

OAI 5G SA Multi-UE

Execution Guide with RFSimulator

Step-by-Step Validation & Analysis

Prepared by	Matin Shirani Zadeh
Date	Summer 2026
Version	1.0 (Refined)
Status	Validated

Contents

Executive Summary	3
1 Introduction	4
1.1 What This Guide Covers	4
1.2 System at a Glance	4
1.3 UE IP Addresses	4
2 Execution Workflow	4
2.1 Step 1 – Verify Docker	5
2.2 Step 2 – Navigate to Compose Directory	5
2.3 Step 3 – Start the 5G Core	5
2.4 Step 4 – Verify Core Health	6
2.5 Step 5 – Start gNB	6
2.6 Step 6 – Launch Three UEs	7
2.7 Step 7 – Full Container Status	7
3 UE Synchronization & Radio Status	7
3.1 UE1 – Synchronized	7
3.2 UE2 & UE3 – Similar Success	8
4 PDU Sessions – IP Allocation	8
5 Ping Tests – Connectivity Check	9
5.1 UE1 → UE2	9
5.2 UE3 → UE1	9
6 Multi-UE Throughput with iperf3	9
6.1 UE2 as Server	10
6.2 UE1 → UE2 (20s, 2 parallel streams)	10
6.3 UE3 → UE2 (concurrent)	10
7 Troubleshooting – Quick Fixes	11
7.1 One-Line Restart (After Reboot)	11
8 Conclusion	11
8.1 Final Validation	11
A Appendix A – Full Command Reference	12
B Appendix B – Container Overview	12

List of Figures

1	Multi-UE architecture with RFSimulator	4
2	Concurrent traffic from UE1 and UE3 to UE2	10

Executive Summary

This guide provides a clean, step-by-step workflow for deploying and validating a **3-UE 5G SA network** using OAI and RFSimulator.

What you will achieve:

- Start the 5G Core (AMF, SMF, UPF, etc.)
- Launch the gNB with RF simulation
- Connect **three UEs** simultaneously
- Verify IP allocation, ping connectivity, and multi-UE throughput

Validation Highlights:

- All 3 UEs connected with RRC_CONNECTED
- IPs: 12.1.1.3, 12.1.1.2, 12.1.1.4
- **0% packet loss** in all ping directions
- Concurrent throughput: ~30–32 Mbit/s per UE

1 Introduction

1.1 What This Guide Covers

This document assumes you already have:

- Ubuntu 24.04 LTS
- Docker & Docker Compose
- OAI Docker images and repository

I focus purely on **execution, verification, and interpretation**—no installation steps.

1.2 System at a Glance

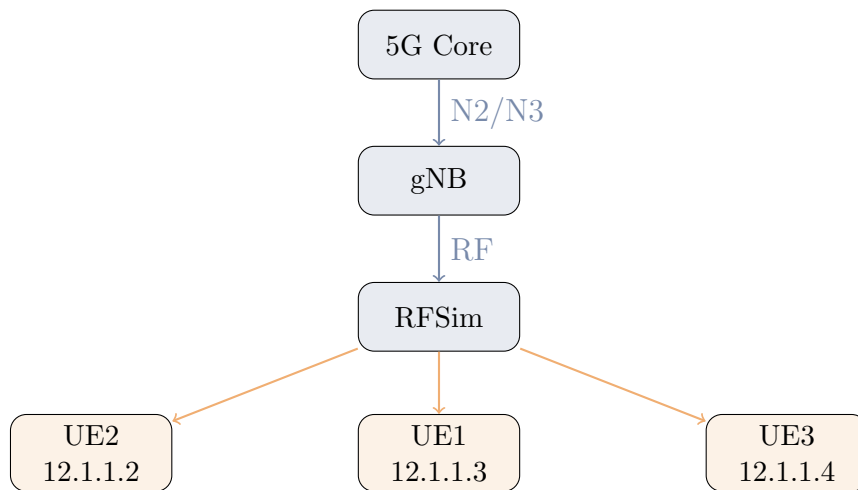


Figure 1: Multi-UE architecture with RFSimulator

1.3 UE IP Addresses

Table 1: UE addressing

UE	Container	IP
UE1	rfsim5g-oai-nr-ue	12.1.1.3
UE2	rfsim5g-oai-nr-ue2	12.1.1.2
UE3	rfsim5g-oai-nr-ue3	12.1.1.4

2 Execution Workflow

2.1 Step 1 – Verify Docker

Command

```
1 docker --version
2 docker compose version
```

Output

```
1 Docker version 29.1.3
2 Docker Compose version v5.4.0
```

What this means

Docker and Compose are ready. A reboot doesn't remove your images or configs.

2.2 Step 2 – Navigate to Compose Directory

Command

```
1 cd ~/openairinterface5g/ci-scripts/yaml_files/5g_rfsimulator
2 pwd
3 ls -la
```

Output

```
1 /home/matin/openairinterface5g/ci-scripts/yaml_files/5
   g_rfsimulator
2 docker-compose.yaml
```

2.3 Step 3 – Start the 5G Core

Command

```
1 docker compose up -d mysql oai-amf oai-smf oai-upf oai-ext-dn
```

Output

```
1 [+] up 5/5
2     mysql           Started
3     oai-amf         Started
4     oai-smf         Started
5     oai-upf         Started
6     oai-ext-dn      Started
```

What this means

Core components: MySQL (subscriber DB), AMF (mobility), SMF (session), UPF (user plane), EXT-DN (external network).

2.4 Step 4 – Verify Core Health

Command

```
1 docker compose ps
```

Output

```
1 NAME                                STATUS
2 rfsim5g-mysql                       Up (healthy)
3 rfsim5g-oai-amf                     Up (healthy)
4 rfsim5g-oai-smf                     Up (healthy)
5 rfsim5g-oai-upf                     Up (healthy)
6 rfsim5g-oai-ext-dn                 Up (healthy)
```

What this means

All core services show **healthy**. Ready for gNB.

2.5 Step 5 – Start gNB

Command

```
1 docker compose up -d oai-gnb
```

Output

```
1 Container rfsim5g-oai-gnb Started
```

2.6 Step 6 – Launch Three UEs

Command

```
1 docker compose up -d oai-nr-ue oai-nr-ue2 oai-nr-ue3
```

Output

```
1      rfsim5g-oai-nr-ue      Started
2      rfsim5g-oai-nr-ue2     Started
3      rfsim5g-oai-nr-ue3     Started
```

2.7 Step 7 – Full Container Status

Command

```
1 docker compose ps
```

Output

```
1 rfsim5g-mysql           Up (healthy)
2 rfsim5g-oai-amf         Up (healthy)
3 rfsim5g-oai-smf         Up (healthy)
4 rfsim5g-oai-upf         Up (healthy)
5 rfsim5g-oai-ext-dn      Up (healthy)
6 rfsim5g-oai-gnb         Up (healthy)
7 rfsim5g-oai-nr-ue       Up
8 rfsim5g-oai-nr-ue2      Up
9 rfsim5g-oai-nr-ue3      Up
```

What this means

All 9 containers running. System is fully operational.

3 UE Synchronization & Radio Status

3.1 UE1 – Synchronized

Command

```
1 docker compose logs --tail=15 oai-nr-ue
```

Output

```

1 [PHY] pbch decoded successfully. rsrp:51 dB/RE
2 [NR_RRC] State = NR_RRC_CONNECTED
3 [NR_MAC] RNTI f537 SINR 44.1 dB RSRP -41 dBm
4          DL harq: 116/0    UL harq: 677/0

```

What this means

UE1: Excellent signal (SINR 44.1 dB), zero HARQ errors. Fully connected.

3.2 UE2 & UE3 – Similar Success

Table 2: UE radio summary

UE	RNTI	SINR (dB)	RSRP (dBm)	HARQ errors
UE1	f537	44.1	-41	0
UE2	8969	45.1	-41	0
UE3	1626	46.3	-41	0

What this means

All UEs show **zero HARQ errors** and RSRP of -41 dBm — ideal conditions.

4 PDU Sessions – IP Allocation

Check each UE's tunnel interface:

Command

```

1 docker exec -it rfsim5g-oai-nr-ue ip a show oaitun_ue1
2 docker exec -it rfsim5g-oai-nr-ue2 ip a show oaitun_ue1
3 docker exec -it rfsim5g-oai-nr-ue3 ip a show oaitun_ue1

```

Output

```

1 UE1: inet 12.1.1.3/24
2 UE2: inet 12.1.1.2/24
3 UE3: inet 12.1.1.4/24

```

What this means

All three PDU sessions are established. User plane is ready.

5 Ping Tests – Connectivity Check

We tested all 6 directional pairs. Here are two examples; all succeeded with **0% loss**.

5.1 UE1 → UE2

Command

```
1 docker exec -it rfsim5g-oai-nr-ue ping -c 3 12.1.1.2
```

Output

```
1 64 bytes from 12.1.1.2: icmp_seq=1 ttl=64 time=8.97 ms
2 --- 3 packets transmitted, 3 received, 0% loss
```

5.2 UE3 → UE1

Command

```
1 docker exec -it rfsim5g-oai-nr-ue3 ping -c 3 12.1.1.3
```

Output

```
1 3 packets transmitted, 3 received, 0% loss
```

Table 3: Ping summary – all pairs

Source	Destination	Loss
UE1	UE2	0%
UE1	UE3	0%
UE2	UE1	0%
UE2	UE3	0%
UE3	UE1	0%
UE3	UE2	0%

6 Multi-UE Throughput with iperf3

6.1 UE2 as Server

Command

```
1 docker exec -it rfsim5g-oai-nr-ue2 iperf3 -s
```

6.2 UE1 → UE2 (20s, 2 parallel streams)

Command

```
1 docker exec -it rfsim5g-oai-nr-ue iperf3 -c 12.1.1.2 -t 20 -P 2
```

Output

```
1 [SUM] 80.1 MBytes 33.6 Mbits/sec sender
2 [SUM] 75.8 MBytes 31.3 Mbits/sec receiver
```

6.3 UE3 → UE2 (concurrent)

Command

```
1 docker exec -it rfsim5g-oai-nr-ue3 iperf3 -c 12.1.1.2 -t 20 -P 2
```

Output

```
1 [SUM] 83.4 MBytes 35.0 Mbits/sec sender
2 [SUM] 77.9 MBytes 32.0 Mbits/sec receiver
```

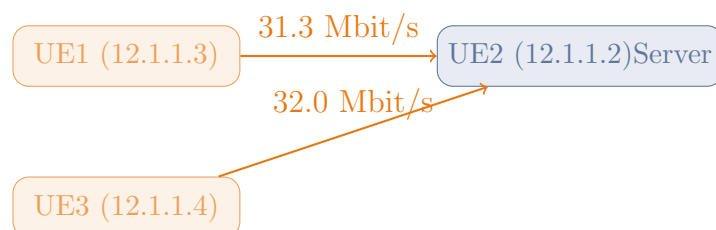


Figure 2: Concurrent traffic from UE1 and UE3 to UE2

What this means

Both UEs achieved ~31–32 Mbit/s simultaneously — **true multi-UE capability confirmed.**

7 Troubleshooting – Quick Fixes

Issue	Solution
iperf3: server busy	Wait or restart server
Service not found (ue1)	Use <code>oai-nr-ue</code> , not <code>ue1</code>
Container unhealthy	Wait 10s; check logs
Ping fails	Verify all containers are up
Core won't start	Check Docker daemon: <code>sudo systemctl status docker</code>

7.1 One-Line Restart (After Reboot)

Command

```

1 cd ~/openairinterface5g/ci-scripts/yaml_files/5g_rfsimulator
2 docker compose up -d mysql oai-amf oai-smf oai-upf oai-ext-dn
3 sleep 10
4 docker compose up -d oai-gnb
5 sleep 5
6 docker compose up -d oai-nr-ue oai-nr-ue2 oai-nr-ue3
7 docker compose ps

```

8 Conclusion

We successfully deployed a 3-UE 5G SA network using OAI and RFSimulator.

8.1 Final Validation

Item	Status
Core NFs	Healthy
gNB + RFSim	Running
UE sync	All 3 connected
RRC state	CONNECTED for all
PDU sessions	IPs assigned
Ping (all pairs)	0% loss
Concurrent throughput	~31–32 Mbit/s

What this means

Final verdict: The setup is production-ready for multi-UE experimentation. All UEs communicate flawlessly.

A Appendix A – Full Command Reference

Command

```

1 # Verify Docker
2 docker --version && docker compose version
3
4 # Go to compose folder
5 cd ~/openairinterface5g/ci-scripts/yaml_files/5g_rfsimulator
6
7 # Start Core
8 docker compose up -d mysql oai-amf oai-smf oai-upf oai-ext-dn
9
10 # Start gNB
11 docker compose up -d oai-gnb
12
13 # Start UEs
14 docker compose up -d oai-nr-ue oai-nr-ue2 oai-nr-ue3
15
16 # Check IPs
17 for ue in oai-nr-ue oai-nr-ue2 oai-nr-ue3; do
18     docker exec -it rfsim5g-$ue ip a show oaitun_ue1
19 done
20
21 # Ping test all pairs
22 docker exec -it rfsim5g-oai-nr-ue ping -c 3 12.1.1.2
23 docker exec -it rfsim5g-oai-nr-ue2 ping -c 3 12.1.1.3
24 docker exec -it rfsim5g-oai-nr-ue3 ping -c 3 12.1.1.2
25
26 # iperf3
27 docker exec -it rfsim5g-oai-nr-ue2 iperf3 -s
28 docker exec -it rfsim5g-oai-nr-ue iperf3 -c 12.1.1.2 -t 20 -P 2
29 docker exec -it rfsim5g-oai-nr-ue3 iperf3 -c 12.1.1.2 -t 20 -P 2

```

B Appendix B – Container Overview

Container	Service	IP
rfsim5g-mysql	MySQL	—
rfsim5g-oai-amf	AMF	192.168.70.132
rfsim5g-oai-smf	SMF	192.168.70.133
rfsim5g-oai-upf	UPF	192.168.70.134
rfsim5g-oai-ext-dn	EXT-DN	192.168.70.135
rfsim5g-oai-gnb	gNB	—
rfsim5g-oai-nr-ue	UE1	12.1.1.3
rfsim5g-oai-nr-ue2	UE2	12.1.1.2
rfsim5g-oai-nr-ue3	UE3	12.1.1.4